SharePoint Document Storage for Tax Document Routing — Microsoft 365 Admin Handoff

Prepared by: Tax Document Routing engineering team (Alamaticz Solutions) **Audience:** the Microsoft 365 / Entra / SharePoint administrators who will configure access **Time needed:** about 30–45 minutes **Document status:** ready to action. The application code is built and unit-tested; it is waiting only on the items in this document.

---

1. Why we are asking — in plain language

What the application is. Tax Document Routing receives tax documents (K-1s, Form 8308, amended returns and similar PDFs), checks them, and files them into the correct folders so tax staff and clients can find them.

What the problem is. The application needs a secure, company-controlled place to keep those files. The original process used Box. The business has decided the files should live in **Microsoft SharePoint**, inside your own Microsoft 365 tenant, under your own retention, security and compliance policies.

What we need from you. For the application to read and write files in SharePoint, a Microsoft 365 administrator must (a) give the application a *limited* identity that is allowed to reach **one SharePoint document library and nothing else**, and (b) hand us the identifiers for that library. Only an administrator can grant this; we cannot do it ourselves.

Why it is safe.

- The application gets access to **one site only** (`Sites.Selected`). It cannot see any other SharePoint site, anyone's OneDrive, mailboxes, Teams chats or files.
- It is granted only the two narrow roles **read** and **write**. It is never given owner or full-control.
- No user signs in for this; it is a service connection. The portal records *which person* triggered each upload in its own audit log.
- Every upload is permission-checked, de-duplicated and audited inside the application **before** it reaches Microsoft.
- You can switch it off at any time (Section 8) without our involvement.

What it will store. Only PDF tax documents that staff upload (and the routed copies the application files from them), at most **2 MB per file**. It does not store system logs, temporary processing data or AI intermediates.

---

2. What you will do, at a glance

Step	What	Who	Approx. time
1	Choose or create the SharePoint site and document library	SharePoint Administrator	10 min

Step	What	Who	Approx. time
2	Confirm / create the two app registrations (read app + write app) and a client secret for the write app	Entra Application Administrator	10 min
3	Add the `Sites.Selected` permission to both apps and grant admin consent	Global Administrator, or Privileged Role Admin + Cloud Application Admin	5 min
4	Grant each app its role (`read` / `write`) on the one site	SharePoint Administrator (needs `Sites.FullControl.All` temporarily)	10 min
5	Look up the Site ID and Drive ID	Any account that can read the site	5 min
6	Confirm the site hardening checklist	SharePoint Administrator	5 min
7	Send us the values in Section 4 through a secure channel	—	—

---

3. Before you start — the two apps

The application uses **two separate Microsoft Entra app registrations** so that a compromise of the read path can never become a write:

App	Purpose	Our setting names	Role it needs on the site
Read app	List, read and download files	`GRAPH_CLIENT_ID` / `GRAPH_CLIENT_SECRET`	`read`
Write app	Upload files	`GRAPH_WRITE_CLIENT_ID` / `GRAPH_WRITE_CLIENT_SECRET`	`write`

Both belong to the **same tenant**. We are asking to **reuse** the two registrations already used for other Microsoft Graph integrations in your organization (for example the Teams-meeting integration used by Project Governance), so that you do not have to maintain new credentials.

Please tell us which is true for each app:

- ☐ The read app already exists and we should reuse it. Name: _____
- ☐ The write app already exists and we should reuse it. Name: _____
- ☐ The app does not exist yet — please create it (Step 2).
- ☐ Your policy requires a separate registration for this application instead of reusing — tell us and we will use new ones.

> If you would rather use a single app for everything, it works, but it puts `write` on the same credential as `read`. We do not recommend it.

---

4. What we need back

Please return these through a **secure channel** (password-manager share or secrets vault — **not** chat or plain email).

#	Value	What it looks like	Where it comes from	Secret?
1	**Tenant (directory) ID**	GUID	Entra admin center → Overview	No
2	**Site ID**	`contoso.sharepoint.com,<guid>,<guid>`	Step 5, call A	No
3	**Drive (document library) ID**	`b!…` (long string)	Step 5, call B	No
4	**Read app** Application (client) ID	GUID	Step 2	No
5	**Read app** client secret **value**	~40 characters, shown once	Step 2	**Yes**
6	**Write app** Application (client) ID	GUID	Step 2	No
7	**Write app** client secret **value**	~40 characters, shown once	Step 2	**Yes**
8	Site URL and library name (for our records)	`https://…/sites/…`	Step 1	No

Please also confirm in writing:

- ☐ Application permissions are **admin-consented** (Step 3)
- ☐ Site access is **granted per app** with only the roles in Step 4, and any temporary high-privilege permission used to grant it has been **removed**
- ☐ The Step 6 hardening items are checked (or exceptions listed)
- ☐ The tenant is the **commercial** cloud (not GCC High / DoD)

> **#5 and #7 are credentials.** Send the secret **value**, not the secret ID. If a secret value was not copied when it was created, create a new secret.

---

5. Step-by-step

Step 1 — Choose or create the site and library

Recommended: a **dedicated site collection** (for example "Tax Document Routing") containing **one document library** (for example "Tax Documents"). Site-level access is the boundary, so nothing else should live beside these documents.

1. SharePoint admin center → **Sites** → **Active sites** → **Create**.
2. Choose **Team site** (or Communication site) and give it a name.
3. Add the owners/members who should see tax documents (see Step 6).

4. Open the site and note the **site URL** and the **library name**.

Do not reuse a busy team site. The access in Step 4 is at *site* level, so everything in the site becomes reachable by the application.

Step 2 — Create the apps (skip any that already exist)

Entra admin center → **Identity** → **Applications** → **App registrations** → **New registration**.

For each app to create:

1. **Name:** `Tax Doc Routing – Graph Read` and `Tax Doc Routing – Graph Write` (or your naming standard).
2. **Supported account types:** *Accounts in this organizational directory only* (single tenant).
3. **Redirect URI:** leave blank.
4. Select **Register**. Copy the **Application (client) ID** and the **Directory (tenant) ID**.
5. **Certificates & secrets** → **New client secret.** Description `tax-doc-routing backend`; expiry per your policy (12–24 months). **Copy the secret value immediately** — Entra shows it only once.

(A certificate instead of a secret is also acceptable and preferable if your policy requires it — tell us and we will adapt.)

Step 3 — Add the permission and grant admin consent (on both apps)

On **each** app:

1. **API permissions** → **Add a permission** → **Microsoft Graph** → **Application permissions.**
2. Search for `Sites.Selected` and tick it → **Add permissions**.
3. Select **Grant admin consent for {your tenant}**. The status must read **Granted**.

> **`Sites.Selected` grants nothing by itself.** It only allows the app to *be given* access to specific sites. Until Step 4, the app has **zero** access to SharePoint.
>
> We are deliberately **not** requesting `Sites.ReadWrite.All`, `Files.ReadWrite.All`, or any tenant-wide file permission.

Step 4 — Grant each app its role on the one site

This is the step that actually gives access. Microsoft requires the `Sites.FullControl.All` permission to make this call, so it must be done by a SharePoint Administrator or higher.

Option A — Microsoft Graph Explorer (sign in as the administrator; consent to `Sites.FullControl.All` for Graph Explorer when prompted):

```
POST https://graph.microsoft.com/v1.0/sites/{site-id}/permissions
Content-Type: application/json

{
  "roles": ["read"],
  "grantedToIdentities": [{
    "application": {
      "id": "<READ_APP_CLIENT_ID>",
      "displayName": "Tax Doc Routing – Graph Read"
    }
  }]
}
```

Repeat the call with `"roles": ["write"]` and the **write** app's client ID and display name.

Option B — PnP PowerShell: `Grant-PnPAzureADAppSitePermission` with `-Permissions Read` for the read app and `-Permissions Write` for the write app. *(Please confirm the cmdlet and parameters against your installed PnP.PowerShell version.)*

Allowed roles are `read`, `write`, `owner`, `fullcontrol`. **Use only `read` and `write`.** Do **not** grant `owner` or `fullcontrol`.

Afterwards: remove any consent you added only to perform this step (for example `Sites.FullControl.All` for Graph Explorer), so no standing full-control permission remains.

Verify: `GET https://graph.microsoft.com/v1.0/sites/{site-id}/permissions` should list exactly two entries — the read app (`read`) and the write app (`write`).

> **Tighter option for your security team:** Microsoft also offers `Files.SelectedOperations.Selected`, `Lists.SelectedOperations.Selected` and `ListItems.SelectedOperations.Selected`, which scope an app to a single library, list, folder or file. These break permission inheritance on the resource, so mind SharePoint's unique-permission limits. We recommend the dedicated-site + `Sites.Selected` approach above; tell us if you prefer narrower and we will adapt.

Step 5 — Look up the two IDs we need

In Graph Explorer (any account that can read the site) or with `curl` and a token:

```
# A. Site ID  (replace host and site name)
GET https://graph.microsoft.com/v1.0/sites/contoso.sharepoint.com:/sites/TaxDocumentRouting
  -> "id": "contoso.sharepoint.com,<guid>,<guid>"          # value #2

# B. Drive (document library) ID
GET https://graph.microsoft.com/v1.0/sites/{site-id}/drives
  -> pick the library by "name"; its "id" is "b!…"         # value #3
```

Please send the **exact strings returned**. We do not construct or guess these IDs.

Step 6 — Site hardening (please confirm each, or list exceptions)

Tax documents identify entities and taxpayers, so the site should sit inside your normal compliance posture:

- ☐ **External sharing disabled** on this site
- ☐ Site covered by your **retention / DLP** policies as appropriate
- ☐ **Versioning on** for the library — the application relies on SharePoint keeping prior versions
- ☐ **Recycle-bin retention** noted (how long deleted files can be restored): _____ days
- ☐ **Sensitivity labels:** please do **not** auto-apply an *encryption* label to this library. Microsoft does not support replacing the content of a labelled file with app-only authentication, so replace operations would fail
- ☐ Site owners / members limited to people who should see tax documents (the app's access does not replace normal SharePoint permissions for humans)
- ☐ Storage quota is sufficient; **blocked file types** and maximum file size noted
- ☐ Any tenant **file-name or path-length restrictions** beyond SharePoint's defaults noted

If your organization requires a HIPAA / BAA / privacy review for a new place that stores tax documents, please flag it now, before go-live.

---

6. What we will test after you hand back the values

1. The backend obtains a token for each app (`login.microsoftonline.com`, client-credentials, scope `https://graph.microsoft.com/.default`).
2. **Read app:** `GET /drives/{drive-id}/root/children` succeeds; the same call against any *other* site is refused (`403`) — this proves the one-site boundary.
3. **Write app:** uploads one small test PDF into the library, replays the same request (it must **not** create a duplicate), and reads it back.
4. We confirm the file appears with the **write app** as "Modified by".

Troubleshooting: `403` usually means Step 4 was not applied to that app yet (or has not propagated). `401` usually means the wrong secret was supplied (the secret *value*, not the secret ID).

---

7. What the application does and does not do

The app **can**	The app **cannot**
List, read and download files in **one** document library on **one** site	Touch any other SharePoint site, OneDrive, mailbox, Teams chat or file
Upload new PDF files (max 2 MB each) to that library, and only with the *write* role	Change site permissions, sharing settings or library settings
Read a file's metadata and version history	Act as, or see the identity of, any signed-in user

The app **can**	The app **cannot**
—	Delete files or folders (no delete operation exists in the application)

Files are attributed to the **application** in SharePoint ("Modified by" shows the app). The identity of the person who triggered each upload is recorded in the application's own audit trail.

---

8. Revoking access later

Any one of these disables the integration:

- `DELETE https://graph.microsoft.com/v1.0/sites/{site-id}/permissions/{permission-id}` — removes that app's access to the site (the cleanest option)
- Revoke admin consent for `Sites.Selected` on the app
- Delete the client secret / certificate
- Delete the app registration

---

9. Appendix — every item and why it exists

Item	Layer	Grants	Without it	Scoped by
`Sites.Selected` (application), read app	Graph / SharePoint	Nothing until Step 4	App cannot be given site access	Step 4 grant
`Sites.Selected` (application), write app	Graph / SharePoint	Nothing until Step 4	App cannot be given site access	Step 4 grant
Admin consent	Entra	Activates the scope	Scope stays inert	—
Site role `read` → read app	SharePoint	List, read, download	App cannot show or open documents	one site
Site role `write` → write app	SharePoint	Upload files	App cannot store documents	one site
Site ID + Drive ID	—	Tells the backend which library to use	Backend has no target	—
Client secret / certificate	Entra	Lets the backend prove it is the app	No token, no access	expiry + secret store
Versioning + recycle bin	SharePoint	Prior versions and restore	No undo for a replaced file	library / site

How the values are used by the application (for your records; you do not set these):

Value	Application setting
Tenant ID	`GRAPH_TENANT_ID`

Value	Application setting
Read app client ID / secret	`GRAPH_CLIENT_ID` / `GRAPH_CLIENT_SECRET`
Write app client ID / secret	`GRAPH_WRITE_CLIENT_ID` / `GRAPH_WRITE_CLIENT_SECRET`
Site ID	`SHAREPOINT_SITE_ID`
Drive ID	`SHAREPOINT_DRIVE_ID`
Auth mode (we set this)	`SHAREPOINT_AUTH_MODE=oauth_client_credentials`

---

10. What we have not verified against your tenant

API behaviour was taken from Microsoft Learn (upload limits, the site-permission grant call and the permission it requires, role names). We have **not** confirmed your tenant's SharePoint limits, naming restrictions, retention settings, sensitivity-label policies or PnP cmdlet versions — those are the items marked “please confirm” above. Nothing in the application has been run against a live tenant yet; the checks in Section 6 are the first live run.

---

11. Reply template

> Tenant ID: _____
> Site URL / library name: _____ / _____
> Site ID: _____
> Drive ID: _____
> Read app — name / client ID: _____ / _____ (secret sent via: _____)
> Write app — name / client ID: _____ / _____ (secret sent via: _____; expires: _____)
> Admin consent granted: yes / no
> Roles granted (read → read app, write → write app) and temporary permission removed: yes / no
> Recycle-bin retention: _____ days; versioning on: yes / no; external sharing off: yes / no
> Exceptions / notes: _____